

Modeling and Predicting Latency Within the Operating Regions of Java Virtual Threads: A Queueing Analysis of Blocking Resource Transformation

1st Bui Quang Huy
Software Engineering
FPT University
Ha Noi, Vietnam
huybqhe180599@fpt.edu.vn

2nd Bui Minh Nhat
Software Engineering
East Asia University of Technology
Ha Noi, Vietnam
20220694@eaut.edu.vn

Abstract

Modern server applications are increasingly constrained by waiting rather than computation. Java Virtual Threads are widely reported to improve scalability under blocking workloads, yet the mechanism behind this improvement remains poorly understood. Existing studies remain largely empirical: no prior work provides an analytical framework that predicts *when* and *why* the two execution models diverge. We formalize Platform Threads (PT) and Virtual Threads (VT) as two execution models over a shared M/M/c service center differing only in the *resource currency* of blocking — native execution contexts (PT) versus heap-resident continuation state (VT). This abstraction yields falsifiable predictions of latency, queue length, and four operating-region boundaries. Validation on a Spring Boot / SQL Server microservice confirms that the abstraction captures the regime transition: a data-driven breakpoint locates the onset of systematic model deviation at utilization $\rho \approx 0.70$ for both execution models. The benefits of Virtual Threads are strongly operating-region dependent: negligible at moderate utilization, but substantial near resource saturation (Cohen’s $d > 1.7$, $|\Delta\text{latency}| > 55$ ms at $\rho \geq 0.85$). These findings support regime-aware capacity planning and identify heap metrics — not thread counts — as the appropriate monitoring signal under Virtual Threads.

Keywords — Java Virtual Threads, Project Loom, queueing theory, Erlang-C, M/M/c, performance modeling, blocking workload, operating regions, capacity planning, resource currency.

1 Introduction

1.1 The Problem Is Waiting, Not Computing

Modern server applications are increasingly constrained by waiting rather than computation. I/O-bound microservices spend the majority of a request’s lifetime blocked on downstream resources — in the experimental setting of this paper, a bounded database connection pool. What differs between threading models is not *whether* requests wait, but *what the system pays while they wait*.

We formalize this distinction through the concept of **resource currency**:

Definition 1 (Resource Currency). *The resource currency of blocking is the physical resource whose occupancy is proportional to the inventory of waiting requests under a given execution model. Formally: for execution model M with blocking inventory N , the currency is the resource R such that $\text{occupancy}(R) \propto E[N]$. Under Platform Threads, $R = \text{native OS thread slots (one thread held per waiting request)}$. Under Virtual Threads,*

$R = \text{heap-resident continuation state (one parked continuation per waiting request, carrier thread released)}$.

The *inventory* of waiting requests is identical across models; the *resource currency* in which that inventory is denominated differs fundamentally. This distinction is the central object of study (Fig. 2).

Under Platform Threads, each blocked request holds a native OS thread open for the full duration of the wait. The thread pool therefore acts as a hard concurrency cap: when all threads are in use, new arrivals queue in memory, but the system has already committed its native thread budget. Under Virtual Threads (Project Loom, Java 21+), blocking unmounts the continuation from its carrier thread. The carrier thread is released immediately to the scheduler, while the waiting state is serialized into heap-resident continuation objects. The inventory of waiting work is unchanged; only its physical substrate differs.

1.2 The Gap in Existing Work

Existing work reports that Virtual Threads improve scalability under blocking workloads [1, 2, 3, 4, 5, 6]. However, these studies remain largely empirical and provide limited analytical understanding of *when* and *why* the two models diverge. No prior model predicts the boundaries of the region where the difference matters, nor formalizes the resource-currency transformation that causes it. Quantitative questions — at what utilization level does VT begin to diverge from PT? How accurately can a classical queueing approximation predict this divergence? Where does the approximation fail and why? — remain unanswered.

This paper is not a new scheduling algorithm. It is not a new runtime. Its goal is to explain observed behavior within a well-defined operating region.

1.3 Research Questions and Hypotheses

We state three research questions and corresponding falsifiable hypotheses **prior to measurement**:

- **RQ1** — How does blocking change resource consumption under each execution model?
- **RQ2** — Can a queueing approximation (M/M/c, Erlang-C) predict latency and queue length within moderate operating regions?
- **RQ3** — Where does the model stop working, and what happens there?

H1 — Blocking changes the resource representation (currency) of waiting under each execution model: PT holds native threads proportionally to blocked inventory; VT parks continuations in heap memory while keeping the carrier pool stable.

H2 — An Erlang-C / M/M/c approximation predicts latency and queue length within a moderate operating region ($\rho \leq \rho^*$), with prediction quality degrading systematically above ρ^* .

H3 — Prediction error tends to increase monotonically as utilization exceeds the saturation breakpoint ρ^* , consistent with progressive violation of model assumptions.

1.4 Contributions

1. **Analytical abstraction of resource transformation** — a formal blocking-currency definition (PT: native thread; VT: heap continuation) over one shared M/M/c service center. *Explains why thread count alone is insufficient to characterize system load under Virtual Threads.*
2. **Operating envelope characterization** — four quantitative, falsifiable regions with boundaries found

[FIGURE 1 — Operating Envelope]

Source:

`analysis/paper/fig1_operating_envelope.png`

Fig. 1. The four-region operating envelope for Java Virtual Threads vs. Platform Threads. Region boundaries are derived from data-driven breakpoint detection ($\rho^* = 0.70$) and effect-size thresholds.

Region I: model reliable, VT/PT latency-indistinguishable. Region II: onset of resource transformation. Region III: continuation accumulation, large VT penalty. Region IV: saturation ($\rho \geq 1$, intentionally unstable).

Figure 1: Operating envelope (placeholder — landscape orientation preferred; full column width).

by data, not chosen by authors. *Reframes the question from “Is VT faster than PT?” to “In which operating region does VT produce meaningful differences?”*

3. **Pre-registered predictive approximation with validated operating envelope** — Erlang-C predictions pre-registered before measurement; agreement analysis quantifies the region where the approximation is reliable ($\text{MAPE} \leq 8\%$ at $\rho \leq 0.70$) and characterizes systematic divergence above ρ^* . *Enables capacity planning in moderate-utilization regimes without full-system benchmarking.*
4. **Design principles for practitioners** — three regime-aware principles derived from the operating envelope specifying appropriate monitoring signals per region. *Translates the operating envelope into actionable guidance for system architects.*

2 Related Work

2.1 Empirical Studies of Java Virtual Threads

The introduction of Project Loom’s Virtual Threads in Java 21 prompted a wave of empirical studies. Lašić et al. [2] evaluated VT throughput in database-driven Spring Boot applications, finding measurable improvements under high concurrency but not providing a predictive model. Beronić et al. [4] surveyed structured concurrency patterns on the JVM and characterized VT as a scalable alternative to reactive frameworks. Hu [5] benchmarked VT against PT in SPECjbb2015 on OpenJDK 25, observing workload-dependent differences without formalizing the boundary. Ponnampalam [6] compared VT and RxJava in a real-time clearing system,

finding complexity-accuracy trade-offs but not addressing the resource-currency mechanism. Šimatović et al. [7] studied GC behavior under high-concurrency VT workloads, noting heap pressure as a concern at high utilization — a finding consistent with our continuation-accumulation abstraction. Charlak et al. [1] compared reactive programming and VT, finding regime-dependent differences without quantifying operating boundaries.

The common limitation across these studies: they *report* performance differences but do not *explain* them analytically. None provides falsifiable, pre-registered predictions; none identifies quantitative model-validity boundaries.

2.2 Queueing Models Applied to Thread Pools

Queueing theory provides established tools for analyzing service systems. M/M/c models and Erlang-C approximations have been applied extensively to classical thread-pool configurations [8], connection pooling [9], and database concurrency [10]. Wu et al. [11] introduced blocking-less queuing (BLQ) as a runtime approach to reducing blocking overhead, but from a compiler perspective rather than an analytical modeling perspective. Poutanen et al. [12] formalized a non-blocking edge API gateway pipeline; their formal model complements our queueing approach but targets a different system class.

The gap: prior queueing work applies to classical thread-pool architectures. No prior work applies these models to the *currency-transforming* behavior of Loom-style virtual threads, where the number of service channels (carriers) becomes decoupled from blocking inventory.

2.3 Performance Measurement Methodology

Reliable performance measurement requires rigorous experimental design. JVM microbenchmarks require warmup, GC stabilization, and replicated measurement [13]. Taipalus [14] provides a systematic review of DBMS performance comparison methodology. Vershinin and Mustafina [15] demonstrate variability in DBMS benchmarks across SQL Server, PostgreSQL, and MySQL configurations. We adopt the replicated-measurement framework with five replicates per operating point, following these methodological standards.

2.4 Gap and Positioning

Existing work evaluates. We explain.

No prior work provides a falsifiable, pre-registered analytical model that (a) formalizes the resource-currency transformation introduced by Virtual Threads, and (b) identifies quantitative boundaries where the model is and is not valid. The contribution is not a faster benchmark, but a principled explanation of *when* and *why* the two execution models diverge.

3 Background: Queueing Theory and Erlang-C

3.1 M/M/c Model

We model the connection pool as an M/M/c queue with:

- Poisson arrival process with rate λ (requests/second)
- Exponential service time with mean $1/\mu$ (seconds)
- c servers (connection pool capacity)
- Infinite waiting room (FCFS discipline)

The offered load (traffic intensity) per server is:

$$\rho = \frac{\lambda}{c \cdot \mu} \quad (1)$$

For stability, $\rho < 1$ is required.

3.2 Erlang-C Formula

The probability that an arriving request must wait (all servers busy) is given by the Erlang-C formula:

$$C(c, a) = \frac{\frac{a^c}{c!} \cdot \frac{c}{c-a}}{\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!} \cdot \frac{c}{c-a}} \quad (2)$$

where $a = \lambda/\mu$ is the offered traffic (Erlangs). Mean waiting time in queue:

$$W_q = \frac{C(c, a)}{c \cdot \mu - \lambda} \quad (3)$$

Mean number in queue (Little’s Law):

$$L_q = \lambda \cdot W_q \quad (4)$$

Mean system latency:

$$W = W_q + \frac{1}{\mu} \quad (5)$$

3.3 Application to Virtual Threads

Key insight: Under Virtual Threads, the *service channels* are not the carrier threads but the connection pool slots. A VT-scheduled request parks its continuation heap-resident when the pool is full; the carrier thread is released. From the queueing perspective, the pool slots remain the c servers; the arrival and service processes are unchanged. The transformation is in the *resource substrate* of the waiting state, not in the queueing geometry.

This is the central observation that permits one M/M/c parameterization to model both execution models — and explains both why it works below ρ^* and why it diverges above it.

4 Methodology

4.1 System Under Test

We instrument a Spring Boot 3.2 microservice connected to a Microsoft SQL Server 2019 instance via synchronous JDBC (HikariCP connection pool, dynamically resizable via `setMaximumPoolSize`). The workload consists of a JPQL query followed by a server-side `WAITFOR DELAY '00:00:00.050'` statement — a **synthetic controlled-blocking workload** in which the connection is genuinely held for the full service duration, making connection hold-time the quantity directly modeled by M/M/c. This construction ensures that $E[S]$ equals connection occupancy time, satisfying the M/M/c server definition. Readers should note that this is a laboratory workload, not a production trace; generalizability to heterogeneous or mixed I/O patterns is a stated limitation (§6.4).

Experiments are executed on a dedicated host (Intel Core i7-12th Gen, 32 GB RAM, Windows 11) to minimize JVM scheduling noise. JVM version: OpenJDK 21.0.2 (Project Loom GA).

4.2 Two-Study Benchmark Design

This paper reports results from **two complementary benchmark programs**, each targeting a distinct aspect of the M/M/c abstraction:

Study 1 — ArrivalSweepBenchmark (primary, ρ -targeted, open-loop). This benchmark implements the M/M/c arrival assumption (Poisson process) by computing $\lambda_{\text{target}} = \rho_{\text{target}} \cdot c \cdot \mu$ and spacing requests using exponential inter-arrival gaps generated by an absolute-deadline dispatcher thread. The connection pool is fixed at $c = 50$ connections. Six utilization levels are evaluated: $\rho \in \{0.30, 0.50, 0.70, 0.85, 0.95, 1.05\}$, where $\rho = 1.05$ is intentionally unstable (outside M/M/c steady-state; included as Region IV boundary evidence only — no PASS/FAIL verdict is assigned). Each operating point is tested under two execution modes (VT and PT), with **5 replicates** per (ρ, mode) cell. Each replicate consists of a **10-second warmup** window followed by a **45-second measurement** window; run order of all $(\text{mode} \times \rho \times \text{replicate})$ cells is randomized (fixed seed = 42) to decorrelate environmental drift from operating point. The achieved arrival rate $\hat{\lambda}$ and inter-arrival coefficient of variation (CV) are recorded per replicate as an empirical check of assumption A1 ($\text{CV} \approx 1 \Rightarrow \text{Poisson-like}$). Queue length $E[N_q]$ is measured directly from `HikariPoolMXBean.getThreadsAwaitingConnection()` sampled every 200 ms — not estimated from the model, avoiding circularity.

Default parameters are configurable via JVM system properties: `-Darrival.pool` (default 50), `-Darrival.reps` (default 5), `-Darrival.warmup` (default 10), `-Darrival.measure` (default 45). The analysis reported in §5 uses the default configuration.

[FIGURE 0 — Conceptual Framework Diagram]

Blocking request $\rightarrow$ *Inventory* $E[N]$
(currency-invariant, Little’s Law)

$\vdash$ *Platform Thread* $\rightarrow$ *Currency: native OS thread occupancy*

$\vdash$ *Virtual Thread* $\rightarrow$ *Currency: heap-resident continuation*

$\rho \leq \rho^*$: *latency-indistinguishable* $\rho > \rho^*$:
continuation accumulation

Fig. 0. The resource-currency transformation. Blocking inventory $E[N]$ is identical under both models (Little’s Law). The physical realization of that inventory — and its consequences for latency — is model-dependent and regime-dependent.

Figure 2: Conceptual framework (placeholder).

Study 2 — PoolSweepValidationBenchmark (secondary, pool-sweep, burst-arrival). This benchmark sweeps connection pool sizes ($\{10, 25, 50\}$ training; $\{100, 200\}$ test) under a **burst-arrival** pattern (requests submitted in a tight loop, λ measured as `total_submitted / submission_window`). It estimates the heap overhead parameter α per queued continuation and validates pool-size predictions via cross-validation. Because the burst-arrival pattern violates the Poisson assumption (A1), this study is treated as a **supplementary** validation of the heap-currency inference rather than the primary latency-vs- ρ validation. Concurrency is set to **4,000** virtual threads with **8 iterations** (2 warmup + 6 measurement) per pool size.

Data source for §5. All latency, queue-length, effect-size, and breakpoint results reported in §5 are derived from Study 1 (ArrivalSweepBenchmark). Heap-memory correlation (Study 2 data) is referenced where noted.

Pre-registration. In both studies, Erlang-C predictions for all operating points are written to CSV files before any measurement cell is executed, satisfying the pre-registration requirement (Framework §6).

4.3 Metrics

- **Mean latency** (ms): mean end-to-end request response time within the measurement window.
- **P99 latency** (ms): 99th-percentile response time (measured only; not predicted by Erlang-C).
- **Queue length** (N_q): mean HikariCP pending-connection count (ground-truth observation from `HikariPoolMXBean`).

- **Heap memory** (MB): mean JVM heap usage sampled every $5 \times 200 \text{ ms} = 1 \text{ s}$.
- **Thread count**: mean JVM live thread count (same sampling interval).
- **Little’s Law check**: $\hat{\lambda} \cdot (\bar{W} - E[S])$ — an internal consistency test for queue-length measurements.
- **Erlang-C predictions**: $\bar{W}$ (mean latency) and $\bar{L}_q$ (queue length) from §3.2, frozen before measurement.

4.4 Statistical Analysis

Prediction error is quantified via MAE, RMSE, and MAPE across the five $\rho < 1$ operating points. Agreement analysis uses:

- **Pearson r** and R^2 (explained variance relative to a flat-line baseline).
- **Paired t-test** (normal residuals, Shapiro–Wilk $p > 0.05$) or **Wilcoxon signed-rank** (non-normal) to test for systematic prediction bias ($n = 5$ per cell; low power — absence of significance is not evidence of absence of bias).
- **Cohen’s d** and **Cliff’s δ** to quantify the practical significance of VT–PT latency differences per ρ level.
- **Automated piecewise regression** for breakpoint detection: ρ^* maximizes SSR improvement; bootstrap 95% CI uses 2,000 resamples.

4.5 Model Assumptions

The Erlang-C / M/M/c approximation rests on the following assumptions. Deviations from these assumptions explain the systematic prediction errors documented in §6.2.

§6.2 references A1–A2 as the primary explanations for model degradation above ρ^* . §6.4 references A3 as the explicit scope boundary for the pinning limitation.

5 Results

5.1 RQ1 — Resource Consumption Is Consistent with Blocking-Currency Transformation

Under low utilization ($\rho \leq 0.50$), both execution models exhibit nearly identical latency despite distinct resource-consumption patterns. PT at $\rho = 0.30$: mean latency 56.4 ms (95% CI ± 0.30 ms, CV = 0.43%); VT at $\rho = 0.30$: 56.5 ms (± 0.46 ms, CV = 0.66%). The difference is statistically and practically negligible.

As utilization crosses the queueing threshold, the same waiting inventory materializes in two distinct physical substrates. Our measurements are consistent with the resource-currency abstraction:

- **Platform Threads**: thread count tracks utilization monotonically (Pearson $r = 0.981$ between Threads_Mean and Rho_Target, Study 1). At $\rho = 0.95$, the measured carrier thread count rose proportionally — consistent with one OS thread held per waiting request.
- **Virtual Threads**: the carrier thread pool remains approximately constant across all ρ levels (Pearson $r = 0.233$ between Threads_Mean and Rho_Target under VT — near zero). At $\rho = 0.95$, the measured carrier count ≈ 138 ($\approx c + \text{OS overhead}$), while queueing estimates via Little’s Law indicate approximately 1,527 continuation objects parked heap-resident ($E[N_q] = \hat{\lambda} \cdot \bar{W}_q$).

The contrast is illustrated in Fig. 3 (resource reallocation): PT’s native thread inventory grows proportionally to load; VT’s carrier inventory remains stable while estimated continuation inventory accumulates.

Caveat: “1,527 continuations” is $E[N_q]$ estimated via Little’s Law from measured $\hat{\lambda}$ and $\bar{W}_q$ — an inferred count consistent with continuation accumulation, not a direct JVM heap measurement. Direct confirmation would require heap profiling (e.g., JVM flight recorder or memory dump). GC/heap profiling is listed as a priority future work item (§6.7).

From Study 2 (PoolSweepValidationBenchmark, pool-sweep, burst-arrival pattern), heap occupancy and pool-induced utilization are positively correlated under VT (Pearson $r = 0.336$ between Heap_Mean_MB and Rho_Target), while under PT they are negatively correlated ($r = -0.547$) — a direction consistent with VT accumulating heap-resident state as queue occupancy grows, while PT’s OS-thread overhead decreases as pool slots saturate. These correlations are from Study 2 measurements and should not be confused with Study 1 (ArrivalSweepBenchmark) results.

Conclusion: Our measurements are consistent with H1. The observed thread-count patterns and heap-correlation directions are consistent with the resource-currency transformation abstraction: blocking inventory under VT is denominated in heap-resident continuation state rather than native OS thread occupancy. Thread count is a potentially misleading monitoring signal under VT at elevated utilization.

5.2 RQ2 — The Approximation Predicts, Within a Region

Claim: Prediction accuracy was acceptable throughout the queue-free operating region, with errors increasing progressively as utilization approached saturation — consistent with H2.

Table 2 presents the full agreement analysis.

Bootstrap 95% CI (latency, all): MAPE = 40.1% [10.4–78.7%]; RMSE = 49.6 ms [15.7–78.1 ms].

Interpretation of the metrics requires care. Aggregate MAPE (59.7% VT, 20.4% PT) is dominated by high- ρ predictions; within the queue-free region ($\rho \leq 0.70$),

Table 1: Model Assumptions and Their Status in This Study

Label	Assumption	Status in This Study
A1	Poisson arrival process (exponential inter-arrival times; $CV = 1$)	Measured inter-arrival $CV > 1$ at high ρ — A1 strained above ρ^*
A2	Exponential service time (single-parameter service distribution)	Synchronous JDBC duration approximately exponential in Region I; distribution shifts near saturation
A3	No carrier thread pinning (no <code>synchronized</code> , <code>JNI</code> , or <code>Object.wait()</code> in workload path)	Enforced by workload design; JDBC driver uses async-compatible locking
A4	Single homogeneous blocking source (synchronous JDBC / SQL Server only)	Enforced; no mixed I/O types in workload
A5	Open-loop workload (arrival rate externally controlled)	Enforced by benchmark design; approximates closed-loop behavior at high concurrency

Table 2: Validation Metrics — Erlang-C vs. Measured Values

Metric	n	MAE	RMSE	MAPE (%)	R^2	r
Mean Latency — VT (ms)	5	41.9	66.2	59.7	-0.277	0.924
Mean Latency — PT (ms)	5	14.5	23.1	20.4	0.182	0.985
Mean Latency — All (ms)	10	28.2	49.6	40.1	-0.102	0.839
Queue Length — VT (req)	5	28.5	48.3	2496.1	-0.236	0.953
Queue Length — PT (req)	5	8.7	16.5	742.3	0.223	0.998

[FIGURE 8 — Resource Reallocation]

Source:

`analysis/paper/fig8_resource_reallocation.png`

Fig. 8. Resource reallocation under increasing utilization. Platform Threads (left axis): native thread count grows proportionally with ρ (Pearson $r = 0.981$). Virtual Threads (right axis): carrier thread count remains approximately stable across all ρ levels ($r = 0.233$), while estimated $E[N_q]$ (inferred via Little’s Law) accumulates heap-resident. The divergence at $\rho \geq 0.85$ is consistent with the resource-currency transformation abstraction.

Figure 3: Resource reallocation (placeholder — dual-axis; annotate $\rho^* = 0.70$).

relative latency error remains $\leq 8\%$ for both models (Fig. 6). Despite high MAPE, Pearson r is strong (0.924 VT, 0.985 PT), confirming that the model captures the *shape* of the latency–utilization relationship across all operating regions.

The negative R^2 values (-0.277 VT, -0.102 all) do not indicate model failure; they indicate that the unconditional mean is a better predictor than the Erlang-C formula over the full ρ range including saturation. This is expected: R^2 is defined relative to a flat baseline, and the

[FIGURE 2 — Latency vs. ρ (gap annotated)]

Source:

`analysis/paper/fig2_latency_vs_rho.png`

Fig. 2. Mean latency vs. target utilization ρ for Virtual Threads (VT) and Platform Threads (PT), with Erlang-C model predictions overlaid. The annotated gap at $\rho \geq 0.85$ marks the region where the model underestimates VT latency — consistent with unmodeled continuation-accumulation overhead. Error bars: 95% CI ($n = 5$ replicates). Dashed vertical line: data-driven breakpoint $\rho^* = 0.70$.

Figure 4: Latency vs. ρ with model overlay (placeholder).

Erlang-C model is not designed for $\rho \geq \rho^*$ — negative R^2 reflects extrapolation outside the model’s validity region rather than poor agreement within Region I. Within Region I ($\rho \leq 0.70$), $MAPE \leq 8\%$ for both models, and the R^2 interpretation does not apply to that subrange. The R^2 gap between PT (0.182) and VT (-0.277) is itself evidence for the resource-transformation thesis: VT incurs additional unmodeled waiting above ρ^* that Erlang-C does not represent (Fig. 4, annotated gap).

Statistical bias tests (Table 3): paired t-test finds no statistically significant bias between prediction and measurement for latency (VT: $t = -1.49$, $p = 0.211$;

[FIGURE 3 — Calibration / Prediction vs. Measurement]

Source: `analysis/paper/fig3_calibration.png`

Fig. 3. Calibration plot: Erlang-C predicted mean latency vs. measured mean latency for all (ρ, mode) pairs. Points above the 1:1 diagonal indicate model underestimation. Filled symbols: $\rho \leq 0.70$ (Region I, quantitative agreement). Open symbols: $\rho > 0.70$ (Regions II–III, model extrapolation). VT points diverge more sharply above ρ^* than PT points, consistent with unmodeled continuation overhead.

Figure 5: Calibration plot (placeholder — square aspect ratio; 1:1 reference line).

PT: $t = -1.23$, $p = 0.285$). Non-normal queue residuals use Wilcoxon signed-rank (VT: $W = 0.0$, $p = 0.250$; PT: $W = 0.0$, $p = 0.250$). No test reaches $\alpha = 0.05$.

Caveat: with $n = 5$ per cell, these tests have low power. Absence of significance is not evidence of absence of bias; it reflects the wide confidence intervals at small sample sizes. These results are consistent with H2 but do not constitute strong confirmation of null bias.

Conclusion: Our results support H2 within Region I ($\rho \leq 0.70$). The approximation is quantitatively useful below $\rho^* \approx 0.70$ and qualitatively informative above it.

5.3 RQ3 — Where the Model Stops Working

The characterization of model validity boundaries proceeds in four steps.

Step 1 — Breakpoint detection. Automated piecewise regression places the onset of systematic deviation at $\rho^* = 0.70$ for both execution models (VT: SSR improvement 99.68%; PT: SSR improvement 98.37%; bootstrap 95% CI for the break: $[0.50, 0.70]$). The boundary was found by the data, not chosen by the authors.

Step 2 — Residual growth beyond ρ^* . Beyond ρ^* , prediction error grows monotonically and directionally:

- VT: relative error +53% at $\rho = 0.85$, +63% at $\rho = 0.95$.
- PT: relative error +18% at $\rho = 0.85$, +40% at $\rho = 0.95$.

The error direction is systematic (the model underestimates measured latency), not noise. VT exhibits substantially larger underestimation than PT. This pattern is *consistent with* the continuation-accumulation interpretation — the Erlang-C model does not account for additional waiting that may arise from heap-resident continuation state (e.g., GC pressure, carrier contention) —

[FIGURE 4 — Residual Plot + Breakpoint CI]

Source: `analysis/paper/fig4_residual.png`

Fig. 4. Residual analysis: relative latency prediction error (%) vs. ρ for VT and PT. Vertical dashed line: automated piecewise breakpoint $\rho^* = 0.70$ (bootstrap 95% CI shaded: $[0.50, 0.70]$). Beyond ρ^* , VT error grows monotonically (+53% at $\rho = 0.85$, +63% at $\rho = 0.95$), while PT error grows more slowly (+18%, +40%). Systematic underestimation direction is consistent with unmodeled continuation-accumulation overhead in VT.

Figure 6: Residual analysis with breakpoint CI (placeholder).

but the precise mechanism is not directly measured and is treated as a hypothesis in §6.2.

Step 3 — Effect sizes across ρ levels. Table 4 reports Cohen’s d and Cliff’s δ for VT vs. PT latency at each utilization level.

Pooled across all ρ : Cohen’s $d = 0.533$, Cliff’s $\delta = 0.152$ (small).

The transition is sharp: at $\rho \leq 0.70$, VT and PT are latency-indistinguishable regardless of effect-size convention. At $\rho \geq 0.85$, VT incurs 55–81 ms higher latency than PT — a large, practically meaningful difference driven by the continuation-accumulation regime.

Step 4 — Operating envelope assembled. The three findings above define the region boundaries illustrated in Fig. 1.

Conclusion: Our results support H3. The model captures the regime transition but systematically underestimates continuation-induced waiting in VT. The negative R^2 at high ρ (VT: -0.277) quantifies exactly the cost component that Erlang-C does not represent, and that gap is itself evidence for the resource-transformation thesis.

6 Discussion

6.1 Why Does the Model Work Below ρ^* ?

The mechanism is Little’s Law. For a system with arrival rate λ and waiting workload W :

$$E[N] = \lambda \cdot W \quad (6)$$

This relationship holds regardless of execution model. Parking a continuation and blocking a thread are the same *queueing* event: both remove one unit of service capacity from the pool’s perspective and add one unit to

Table 3: Statistical Significance of Prediction Error ($n = 5$ per cell)

Comparison	Test	Statistic	p	Significant
Latency VT	Paired t-test	-1.49	0.211	No
Latency PT	Paired t-test	-1.23	0.285	No
Queue VT	Wilcoxon signed-rank	0.00	0.250	No
Queue PT	Wilcoxon signed-rank	0.00	0.250	No

Table 4: VT vs. PT Effect Size by Utilization Level

ρ	VT mean (ms)	PT mean (ms)	$ \Delta \text{ latency} $	Cohen’s d	Cliff’s δ	Magnitude
0.30	56.5	56.4	0.15 ms (0.3%)	0.492	0.400	Medium ¹
0.50	59.1	59.4	0.29 ms	-0.164	-0.120	Negligible
0.70	64.6	64.4	0.19 ms	0.145	0.200	Small
0.85	131.2	75.5	55.6 ms (74%)	1.943	1.000	Large
0.95	207.0	125.7	81.3 ms (65%)	1.707	0.840	Large

the inventory $E[N]$. Below ρ^* , currency choice is invisible to latency — which is why one Erlang-C parameterization fits both execution models.

Lemma 1 (Resource Currency Invariance). *For a system with arrival rate λ and waiting workload W , Little’s Law gives $E[N] = \lambda W$ independently of execution model. The physical realization of N is model-dependent:*

- *PT: N represents native thread occupancy (one OS thread per waiting request).*
- *VT: N represents heap-resident continuation state (one parked continuation per waiting request).*

The waiting time W is therefore currency-invariant, while its physical substrate is not.

Corollary: Below ρ^* , the two execution models are latency-indistinguishable by any queueing observer. Above ρ^* , continuation accumulation introduces waiting that Erlang-C does not model, explaining the systematic underestimation in VT.

This corollary also implies a practical equivalence: in Region I, VT vs. PT is a **design decision that should be made on non-performance grounds** — memory footprint, operational constraints, code complexity — because latency consequences are negligible.

6.2 Why Does Prediction Degrade Above ρ^* ?

The degradation is consistent with violation of assumptions A1 and A2 (§4.5). We advance the following hypotheses — we do not assert causal mechanisms, as testing them is future work:

1. **Burst arrivals (A1 violated).** At high ρ , measured inter-arrival coefficient of variation (CV) exceeds 1 — arrivals are burstier than Poisson. The Erlang-C formula underestimates mean waiting time for super-Poisson arrivals (M[X]/M/c effect).

2. **Service time distribution shift (A2 strained).** At high queue occupancy, HikariCP connection acquisition time becomes the dominant component of service time. The combined distribution (SQL delay + pool acquisition) is no longer well-approximated by a single-parameter exponential.
3. **VT-specific: continuation overhead (hypothesis, not directly measured in this study).** Under VT at high utilization, the heap may accumulate thousands of parked continuations. GC pressure and continuation-state overhead could introduce additional waiting not captured by M/M/c. This mechanism — unique to VT — is consistent with the observed VT-PT divergence in model accuracy at $\rho \geq 0.85$, but is inferred from latency and queue-length data; direct heap profiling or GC-pause instrumentation would be required to confirm it (see §6.7, item 4).

The VT-specific prediction gap ($\text{MAPE}_{\text{VT}} = 59.7\%$ vs. $\text{MAPE}_{\text{PT}} = 20.4\%$) is consistent with hypothesis 3: PT’s blocking substrate (OS threads) is accounted for implicitly in the pool model; VT’s continuation substrate is not.

6.3 Practical Implications and Design Principles

The operating envelope translates directly into actionable guidance for capacity planners and system architects.

DP1: Monitor resource currency, not thread count. Under Virtual Threads, the relevant resource is heap-resident continuation state, not carrier thread pool size. Thread count is a misleading monitoring signal above ρ^* ; heap occupancy and GC pressure are the appropriate metrics in Regions II–III.

DP2: Apply queueing approximation only within the valid operating region. Erlang-C predictions are

Table 5: Operating Envelope Regions

Region	ρ Range	Characterization
I — Queue-free	$\rho \leq 0.70$	Model reliable; VT/PT latency-indistinguishable
II — Resource transformation	$0.70 < \rho \leq 0.85$	Onset of continuation accumulation; directional model only
III — Continuation accumulation	$0.85 < \rho \leq 1.0$	Large VT penalty; heap-driven dynamics; model as warning signal only
IV — Saturation	$\rho \rightarrow 1$	Latency increases rapidly until bounded by timeout or heap limits

quantitatively reliable in Region I ($\rho \leq 0.70$). In Region II ($0.70 < \rho \leq 0.85$), treat model output as directional only. In Region III, use the model as a saturation warning, not a capacity estimate.

DP3: Capacity planning should be regime-aware. The boundary $\rho^* \approx 0.70$ defines the transition from analytical predictability to regime-dependent behavior. Identify the expected operating region first, then select monitoring and scaling strategies accordingly.

6.4 Limitations

Analytical assumptions were intentionally simplified to yield closed-form, falsifiable predictions. The following limitations define the scope of validity and constitute the boundary between what this paper claims and what it does not.

Single DBMS and framework. The experiment uses SQL Server 2019 and Spring Boot 3.2 / Tomcat. Whether the operating-region boundaries shift for PostgreSQL [15], MySQL, or non-relational stores [16] is unknown. The queueing abstraction should generalize — the specific ρ^* value may not.

Single blocking type. The workload uses synchronous JDBC with a fixed delay. Generalizability to network I/O (HTTP, Redis, Kafka) or mixed blocking patterns is untested.

Sample size. Five replicates per cell produces wide confidence intervals at high utilization (CV up to 37.5% at $\rho = 0.95$ for PT). The breakpoint CI [0.50, 0.70] is constrained by the five-level ρ grid; intermediate levels (e.g., $\rho \in \{0.60, 0.75, 0.80\}$) would tighten this interval.

Indirect resource-currency measurement. Continuation counts are estimated via $E[N_q]$ from queueing models (Little’s Law), not from direct JVM heap profiling. The “continuation accumulation” narrative is an inference consistent with the data, not a direct measurement.

Virtual Thread pinning excluded (A3). The experimental workload enforces no **synchronized** blocks or JNI calls. Workloads that trigger carrier thread pinning (legacy **synchronized**, JNI, or `Object.wait()`) invalidate the currency-transformation abstraction: a pinned VT holds both a continuation and a native thread, collapsing the currency distinction. Model behavior under pinning is explicitly out of scope.

JVM version. The Loom scheduler evolves across JDK releases. Results are calibrated for OpenJDK 21.0.2;

future JDK versions may shift operating-region boundaries.

6.5 Threats to Validity

Internal validity. JVM warmup effects are mitigated by a **10-second warmup** window before each measurement cell (Study 1: ArrivalSweepBenchmark default). GC pauses are not explicitly isolated; high- ρ VT measurements may include GC-induced latency spikes. Replication ($n = 5$) and mean-based analysis reduce but do not eliminate this threat. The 10-second warmup is shorter than some JVM benchmark conventions; JIT compilation and connection-pool stabilization effects cannot be fully excluded, though the randomized run order (fixed seed = 42) mitigates systematic drift.

External validity. Results are from a single machine configuration. Cloud or containerized deployments with shared CPU/memory resources may exhibit different ρ^* boundaries due to OS thread scheduling interactions.

Construct validity. Erlang-C is a steady-state model; the benchmark enforces a **45-second measurement window** per cell (Study 1 default). Transient effects at load-step transitions are excluded by the warmup window, but the 45-second window may not fully reach steady state at $\rho \approx 0.95$ where drain times are longest.

6.6 Relation to Prior Work

Prior benchmarking studies report throughput or latency improvements under blocking workloads and answer “Is VT faster?” — a valuable but incomplete question. This paper asks instead: *Why, when, and under what conditions?*

By formalizing the resource-currency transformation analytically and validating falsifiable predictions, we move from empirical observation to mechanistic explanation. The model applicability boundary ($\rho^* \approx 0.70$) and the four-region operating envelope are not findings that prior benchmarks could have produced — they require a predictive abstraction to compare against, not just measurements.

6.7 Future Work

Five extensions that would materially strengthen the generalizability of these results:

Table 6: Monitoring Signal Checklist by Operating Region

Signal	Region I ($\rho \leq 0.70$)	Region II ($0.70 < \rho \leq 0.85$)	Region III ($\rho > 0.85$)
Carrier thread count	× Misleading under VT	×	×
Request queue length	✓ Reliable	✓	✓
Heap occupancy	Optional	✓ Watch	✓ Primary signal
GC pause frequency	Optional	✓ Watch	✓ Primary signal
Erlang-C model output	✓ Reliable	~ Directional only	~ Saturation warning

1. **Sensitivity analysis on service time.** Repeat with blocking durations of 20 ms, 40 ms, 80 ms to test whether ρ^* shifts with mean service time. Expected finding: ρ^* should be relatively stable, which would support model robustness.
2. **Second database.** PostgreSQL with the same workload and load levels would confirm that the envelope is not SQL Server-specific.
3. **Second blocking type.** Network sleep, HTTP, Redis, or Kafka I/O. Even one additional blocking type would permit a limited transferability claim and directly address the generalizability limitation.
4. **Direct GC and heap metrics.** JVM heap occupancy and GC pause frequency across ρ levels would convert the “continuation accumulation” narrative from *inferred* to *directly observed*, resolving the resource-currency measurement gap.
5. **Confidence-interval figures.** Adding 95% CI bands to all latency-vs- ρ figures would visually support the $n = 5$ discussion and simplify interpretation of the effect-size table.

7 Conclusion

This paper formalizes Platform Threads and Virtual Threads as two execution models over a shared M/M/c service center, differing only in the resource currency of blocking: native OS thread occupancy (PT) versus heap-resident continuation state (VT).

Key finding: The blocking inventory $E[N]$ is identical under both models (governed by Little’s Law); the *physical substrate* of that inventory is model-dependent. This currency-invariance of queueing dynamics explains why one Erlang-C parameterization fits both execution models below ρ^* — and why it diverges above it.

Empirical summary:

- At $\rho \leq 0.70$, VT and PT are latency-indistinguishable ($|\Delta\text{latency}| < 0.3$ ms; all effect sizes negligible to small). The Erlang-C approximation achieves MAPE $\leq 8\%$ in this region.
- At $\rho \geq 0.85$, VT incurs 55–81 ms higher mean latency than PT (Cohen’s $d > 1.7$; large effect).

Model accuracy degrades substantially (VT MAPE = 59.7% over all ρ).

- A data-driven breakpoint at $\rho^* = 0.70$ (SSR improvement: VT 99.68%, PT 98.37%; 95% CI [0.50, 0.70]) separates the analytically tractable from the accumulation-driven regime.

Practical take-away: VT’s scalability advantages are strongly regime-dependent. Below $\rho^* \approx 0.70$, VT and PT are performance-equivalent — the choice should be made on other grounds. Above ρ^* , VT’s continuation accumulation generates additional latency not captured by classical queueing models, and heap occupancy / GC pressure — not carrier thread count — become the appropriate monitoring signals.

The resource-currency abstraction provides a principled foundation for regime-aware capacity planning in JVM-based, I/O-bound microservices.

Acknowledgment

(To be added: hardware/lab access, anonymous reviewers.)

References

- [1] D. Charlak, J. Brzeziński, and G. Koziel, “Comparative analysis of reactive programming and Java virtual threads,” *Journal of Computer Sciences Institute*, vol. 39, pp. 154–160, 2026.
- [2] L. Lašić, D. Beronić, B. Mihaljević, and A. Radovan, “Assessing the efficiency of Java virtual threads in database-driven server applications,” in *Proc. 2024 47th MIPRO ICT and Electronics Convention*, 2024.
- [3] P. Pufek, D. Beronić, B. Mihaljević, and A. Radovan, “Achieving efficient structured concurrency through lightweight fibers in Java Virtual Machine,” in *Proc. 2020 43rd Int. Convention on Information, Communication and Electronic Technology (MIPRO)*, 2020, pp. 1629–1634. doi: 10.23919/MIPRO48935.2020.9245253.

- [4] D. Beronić, N. Novosel, B. Mihaljević, and A. Radovan, “On analyzing virtual threads — a structured concurrency model for scalable applications on the JVM,” in *MIPRO Conference*, 2024.
- [5] Z. Hu, “Platform threads versus virtual threads in SPECjbb2015: An empirical study on OpenJDK 25,” Uppsala University: Disciplinary Domain of Science and Technology, 2026.
- [6] S. Ponnampalam, “Evaluation of Virtual Threads and RxJava: A performance and code complexity analysis in a real-time clearing system,” Degree Project in Computing Science and Engineering, Uppsala University, 2025.
- [7] M. Šimatović, H. Markulin, D. Beronić, and B. Mihaljević, “Evaluating memory management and garbage collection algorithms with virtual threads in high-concurrency Java applications,” in *48th ICT and Electronics Convention MIPRO 2025*, 2025.
- [8] B. Pereira and F. M. Carvalho, “Enabling progressive server-side rendering for traditional web template engines with Java virtual threads,” *Software*, vol. 4, no. 3, p. 20, 2025. doi: 10.3390/software4030020.
- [9] T. Cherny-Shahar and A. Yehudai, “MetaFFI-Multilingual indirect interoperability system,” *Software*, vol. 4, no. 3, p. 21, 2025. doi: 10.3390/software4030021.
- [10] L. Phan, M. Wang, G. Wu, W. Dawei, C. Liqun, and L. Jin, “CrossTrace: Efficient cross-thread and cross-service span correlation in distributed tracing for microservices,” arXiv, 2025.
- [11] Q. Wu, R. Li, J. Beard, and L. John, “BLQ: Light-weight locality-aware runtime for blocking-less queuing,” in *Proc. 33rd ACM SIGPLAN Int. Conf. on Compiler Construction (CC ’24)*, 2024. doi: 10.1145/3640537.3641568.
- [12] Poutanen et al., “A formal model of an asynchronous, non-blocking edge API gateway pipeline,” *Preprints.org*, 2026.
- [13] S. Yang et al., “Thread architecture models (TAMs),” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [14] T. Taipalus, “Database management system performance comparisons: A systematic literature review,” *ResearchGate*, 2022.
- [15] I. Vershinin and A. R. Mustafina, “Performance analysis of PostgreSQL, MySQL, Microsoft SQL Server,” in *Proc. 2021 Int. Russian Automation Conference (RusAutoCon)*, 2021. doi: 10.1109/RusAutoCon52004.2021.9537400.
- [16] M. Sagan and M. Plechawska-Wójcik, “Comparative analysis of non-relational databases on the example of Amazon DynamoDB and MongoDB,” *Journal of Computer Sciences Institute*, vol. 39, pp. 108–114, 2026.
- [17] Z. Wu, J. Liang, J. Fu, M. Wang, and Y. Jiang, “PUPPY: Finding performance degradation bugs in DBMSs via limited-optimization plan construction,” in *Proc. 2025 IEEE/ACM 47th Int. Conference*, 2025.
- [18] S. Tavakolisomeh, R. Bruno, and P. Ferreira, “BestGC: An automatic GC selector software,” *ResearchGate*, 2023.
- [19] W. Wnuk and M. Plechawska-Wójcik, “Comparative performance analysis of Express.js and Spring Boot in CRUD-oriented web applications,” *Journal of Computer Sciences Institute*, vol. 39, pp. 176–182, 2026. doi: 10.35784/jcsi.9574.
- [20] M. Serej, K. Kopciński, and J. Smolka, “Comparative performance analysis of Spring Boot and Ktor for a ticket reservation REST API on the JVM,” *Journal of Computer Sciences Institute*, vol. 39, pp. 183–187, 2026.
- [21] V. Solohub and V. Pashkevych, “Comparative analysis of query optimization techniques in modern relational database systems,” *Journal of Computer Sciences Institute*, vol. 39, pp. 132–137, 2026.
- [22] D. C. Ma’soem, “Implementation of event-driven architecture using NATS JetStream for a large-scale online exam answer collection system,” *Journal of Business, Social and Technology*, vol. 7, no. 2, pp. 482–495, 2024.

Appendix A — Supplementary Figures

The following three figures are supplementary — they provide supporting evidence but are not the primary vehicles for the paper’s claims. They may be placed in an online appendix if page limits require.

Word count (body, excluding references and appendix): $\approx$ 6,800 words. Target journal page count: 12–14 pages (JSS/SPE double-column format).

[FIGURE A1 — Bland–Altman Analysis]

Source: `analysis/paper/fig5_bland_altman.png`

Fig. A1. Bland–Altman agreement plot of Erlang- C predicted vs. measured mean latency (all $\rho < 1$ operating points, both models). Horizontal lines: mean bias ± 1.96 SD (limits of agreement). Points within $\rho^* = 0.70$ cluster near zero bias; high- ρ points show systematic positive bias (model underestimates), confirming the regime-transition pattern from §5.

Figure 7: Bland–Altman analysis (placeholder).

[FIGURE A2 — VT vs. PT Side-by-Side Panels]

Source: `analysis/paper/fig6_vt_vs_pt.png`

Fig. A2. Side-by-side latency trajectory comparison for Virtual Threads (left) and Platform Threads (right) across all $\rho \in \{0.30, 0.50, 0.70, 0.85, 0.95, 1.05\}$. Solid lines: measured mean latency ($n = 5$ replicates, error bars: 95% CI). Dashed lines: Erlang- C predictions. Note the divergence between predicted and measured VT latency at $\rho \geq 0.85$, absent in PT.

Figure 8: VT vs. PT side-by-side panels (placeholder).

[FIGURE A3 — Correlation Heatmap]

Source:

`analysis/paper/fig7_correlation_heatmap.png`

Fig. A3. Pearson correlation matrices for Study 1 (ArrivalSweepBenchmark) measurements. Left: Platform Threads — Threads_Mean strongly correlated with Rho_Target ($r = 0.981$), confirming native-thread occupancy scales with load. Right: Virtual Threads — Threads_Mean nearly uncorrelated with Rho_Target ($r = 0.233$), while latency and queue-length metrics remain strongly correlated, consistent with the carrier-pool stability prediction of the resource-currency abstraction.

Figure 9: Correlation heatmap (placeholder).